Tugas Praktikum Sesi 4
Pemrograman Mobile



Dosen Pengampu: Jefry Sunupurwa Asri, S.Kom, M.Kom

Disusun Oleh:

Bagas Yoas Sibagariang

(20230801254)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

TANGERANG

2026

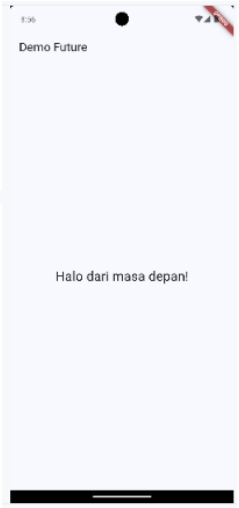
1. Langkah – Langkah Praktikum

a. Mengenal Future dan FutureBuilder (tanpa internet)
Ganti isi lib/main.dart dengan kode berikut. Fungsi ambilSalam() mensimulasikan pengambilan data yang butuh 2 detik.

```dart
import 'dart:convert';
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 4',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const SalamPage(),
    );
  }
}
```

```dart
// ==================== Bagian A ====================
Future<String> ambilSalam() async {
  await Future.delayed(const Duration(seconds: 2));
  return 'Halo dari masa depan!';
}
class SalamPage extends StatefulWidget {
  const SalamPage({super.key});
  @override
  State<SalamPage> createState() => _SalamPageState();
}
class _SalamPageState extends State<SalamPage> {
  late Future<String> _future;
  @override
  void initState() {
    super.initState();
    _future = ambilSalam();
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Future')),
      body: Center(
        child: FutureBuilder<String>(
          future: _future,
          builder: (context, snapshot) {
            if (snapshot.connectionState == ConnectionState.waiting) {
              return const CircularProgressIndicator();
            }
            if (snapshot.hasError) {
              return Text('Error: ${snapshot.error}');
            }
            return Text(snapshot.data!, style: const TextStyle(fontSize: 24));
          },
        ),
      ),
    );
  }
}
```



b. Menyiapkan Paket HTTP

```
→ praktikum_4 git:(main) ✗ flutter pub add http
   Woah! You appear to be trying to run flutter as root.
   We strongly recommend running the flutter tool without superuser privileges.
  /
"http" is already in "dependencies". Will try to update the constraint.
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
  flutter_lints 4.0.0 (6.0.0 available)
  lints 4.0.0 (6.1.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  test_api 0.7.12 (0.7.15 available)
Got dependencies!
6 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
→ praktikum_4 git:(main) ✗
```

c. Model dan Fungsi Pengambil Data
Struktur data dari /users berupa daftar objek JSON dengan kolom seperti id, name, email, phone, dan website. Tambahkan kode berikut di bawah import pada main.dart, dan tambahkan dua import baru di bagian paling atas:

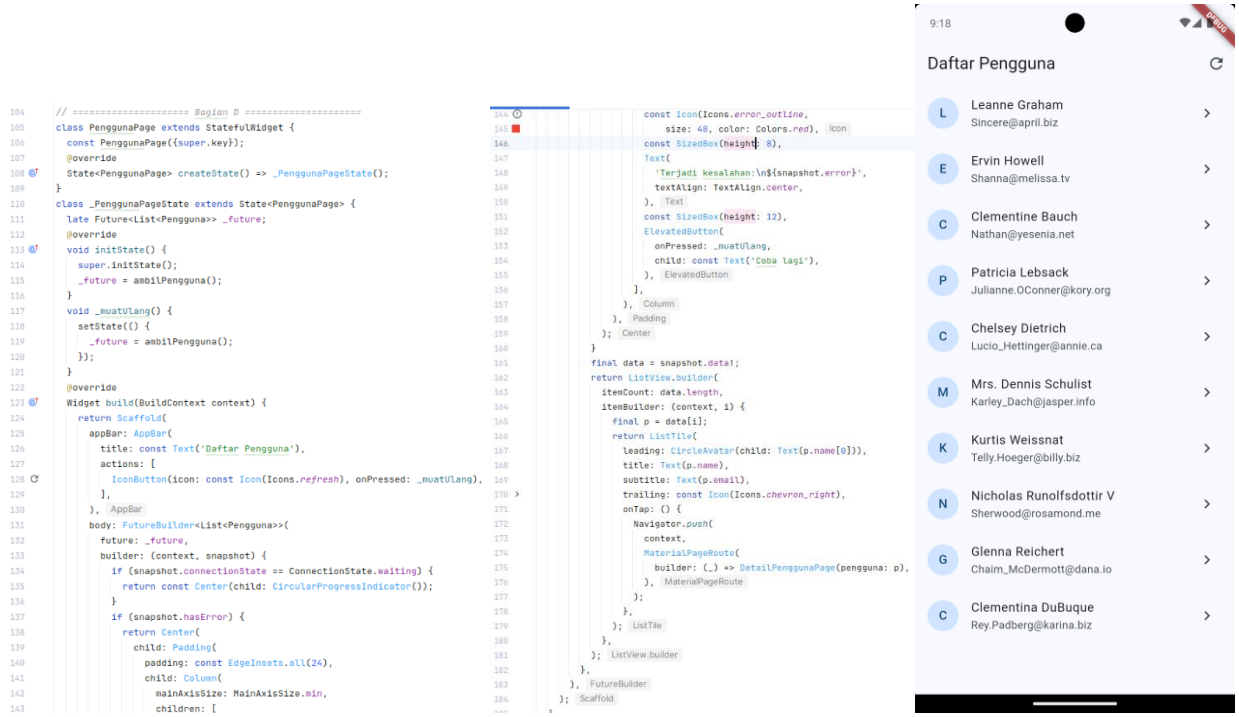
```dart
// ==================== Bagian C ====================
class Pengguna {
  final int id;
  final String name;
  final String email;
  final String phone;
  final String website;

  const Pengguna({
    required this.id,
    required this.name,
    required this.email,
    required this.phone,
    required this.website,
  });

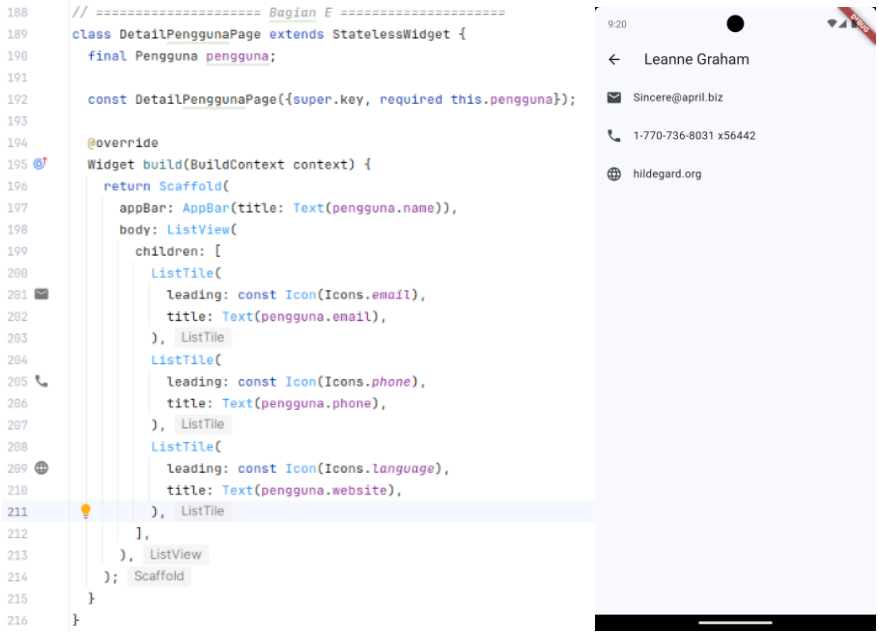
  factory Pengguna.fromJson(Map<String, dynamic> json) {
    return Pengguna(
      id: json['id'] as int,
      name: json['name'] as String,
      email: json['email'] as String,
      phone: json['phone'] as String,
      website: json['website'] as String,
    );
  }
}
```

```dart
Future<List<Pengguna>> ambilPengguna() async {
  final uri = Uri.parse('https://jsonplaceholder.typicode.com/users');
  final response = await http.get(uri).timeout(const Duration(seconds: 10));
  if (response.statusCode != 200) {
    throw Exception('Gagal memuat data (kode ${response.statusCode})');
  }
  final List<dynamic> data = jsonDecode(response.body);
  return data
      .map((e) => Pengguna.fromJson(e as Map<String, dynamic>))
      .toList();
}
```

d. Menampilkan Data dengan FutureBuilder
Tambahkan halaman berikut, lalu ubah home: menjadi const PenggunaPage(). Halaman ini menangani tiga keadaan: memuat, galat (dengan tombol coba lagi), dan data.



e. Halaman Detail



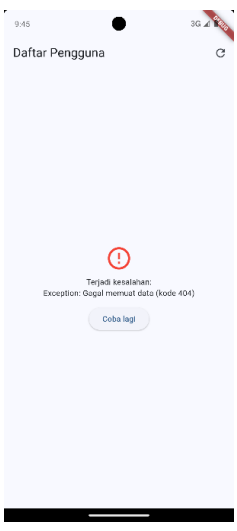
f. Percobaan Penanganan Galat

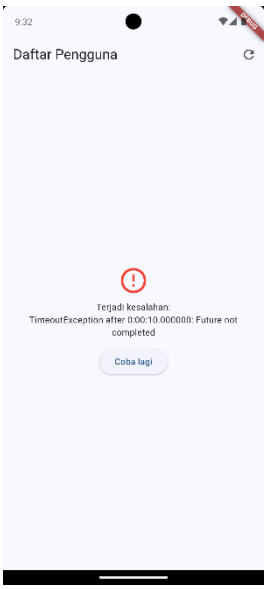
Lakukan dua percobaan berikut dan catat tampilan yang muncul untuk laporan:

1. Ubah URL menjadi /userz (halaman tidak ada). Amati pesan galat dengan kode 404, lalu kembalikan URL yang benar dan tekan Coba lagi.

```
Future<List<Pengguna>> ambilPengguna() async {
  final uri = Uri.parse('https://jsonplaceholder.typicode.com/userz');
  final response = await http.get(uri).timeout(const Duration(seconds: 10));
```

2. Matikan koneksi internet perangkat/emulator lalu tekan tombol refresh. Amati pesan galat jaringan, nyalakan internet kembali, dan tekan Coba lagi





2.Latihan Mandiri

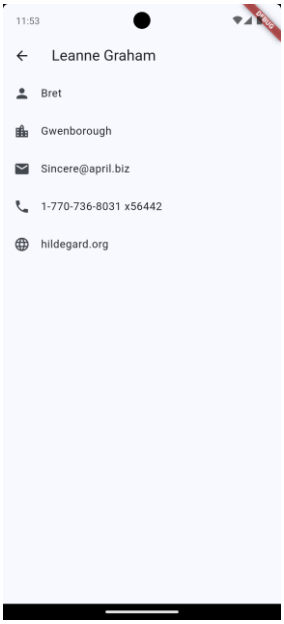
1. Tambahkan kolom username dan kota (data bersarang: json['address']['city']) pada model dan tampilkan di halaman detail.

```dart
class Pengguna {
  final int id;
  final String name;
  final String username;
  final String email;
  final String phone;
  final String website;
  final String kota;

  const Pengguna({
    required this.id,
    required this.name,
    required this.username,
    required this.email,
    required this.phone,
    required this.website,
    required this.kota,
  });

  factory Pengguna.fromJson(Map<String, dynamic> json) {
    return Pengguna(
      id: json['id'] as int,
      name: json['name'] as String,
      username: json['username'] as String,
      email: json['email'] as String,
      phone: json['phone'] as String,
      website: json['website'] as String,
      kota: json['address']['city'] as String, // data bersarang
    );
  }
}
```

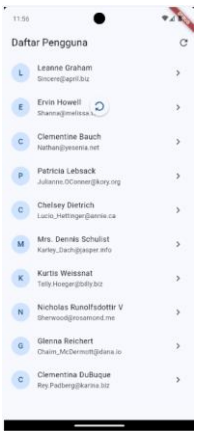
```dart
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text(pengguna.name)),
    body: ListView(
      children: [
        ListTile(
          leading: const Icon(Icons.person),
          title: Text(pengguna.username),
        ), ListTile
        ListTile(
          leading: const Icon(Icons.location_city),
          title: Text(pengguna.kota),
        ), ListTile
```



2. Bungkus ListView dengan RefreshIndicator agar dapat ditarik ke bawah untuk memuat ulang.

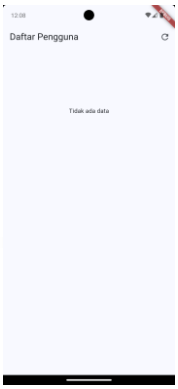
```dart
class PenggunaPage extends StatefulWidget {
  const PenggunaPage({super.key});
  @override
  State<PenggunaPage> createState() => _PenggunaPageState();
}
class _PenggunaPageState extends State<PenggunaPage> {
  late Future<List<Pengguna>> _future;
  @override
  void initState() {
    super.initState();
    _future = ambilPengguna();
  }
  void _muatUlang() {
    setState(() {
      _future = ambilPengguna();
    });
  }
  Future<void> _tarikUntukMuat() async {
    setState(() {
      _future = ambilPengguna();
    });
    try {
      await _future; // tunggu selesai supaya animasi tarik berhenti
    } catch (_) {
      // galat ditampilkan oleh FutureBuilder
    }
  }
```

```dart
final data = snapshot.data!;
return RefreshIndicator(
  onRefresh: _tarikUntukMuat,
  child: ListView.builder(
    physics: const AlwaysScrollableScrollPhysics(),
    itemCount: data.length,
    itemBuilder: (context, i) {
      final p = data[i];
      return ListTile(
        leading: CircleAvatar(child: Text(p.name[0])),
        title: Text(p.name),
        subtitle: Text(p.email),
        trailing: const Icon(Icons.chevron_right),
        onTap: () {
          Navigator.push(
            context,
            MaterialPageRoute(
              builder: (_) => DetailPenggunaPage(pengguna: p),
            ), MaterialPageRoute
          );
        },
      ); ListTile
    },
  ), ListView.builder
); RefreshIndicator
```



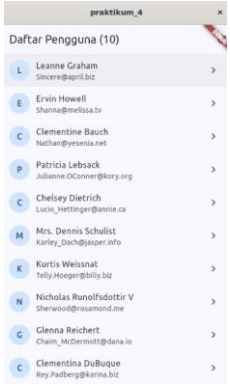
3. Bila daftar kosong, tampilkan teks "Tidak ada data".

```dart
final data = snapshot.data!;
if (data.isEmpty) {
  return RefreshIndicator(
    onRefresh: _tarikUntukMuat,
    child: ListView(
      physics: const AlwaysScrollableScrollPhysics(),
      children: const [
        SizedBox(
          height: 300,
          child: Center(child: Text('Tidak ada data')),
        ), SizedBox
      ],
    ), ListView
  ); RefreshIndicator
}
return RefreshIndicator(
```



4. Tambahkan Text di AppBar yang menampilkan jumlah pengguna setelah data berhasil dimuat.

```dart
appBar: AppBar(
  title: FutureBuilder<List<Pengguna>>(
    future: _future,
    builder: (context, snapshot) {
      if (snapshot.hasData) {
        return Text('Daftar Pengguna (${snapshot.data!.length})');
      }
      return const Text('Daftar Pengguna');
    },
  ), FutureBuilder
  actions: [
    IconButton(icon: const Icon(Icons.refresh), onPressed: _muatUlang),
  ],
), AppBar
body: FutureBuilder<List<Pengguna>>(
  future: _future,
  builder: (context, snapshot) {
```



3. Tugas

Buat aplikasi Daftar Postingan menggunakan endpoint /posts dan /posts/{id}/comments dengan ketentuan:

- Halaman utama menampilkan daftar postingan (judul dan potongan isi) dengan FutureBuilder.
- Mengetuk postingan membuka halaman detail yang menampilkan isi lengkap dan daftar komentar yang diambil dari endpoint terpisah (Future kedua).
- Model Post dan Komentar dengan fromJson; kode pengambil data dipisah dari kode UI.
- Menangani status loading dan galat pada kedua halaman, termasuk tombol coba lagi.
- Kumpulkan: tangkapan layar (daftar, detail, dan tampilan galat) serta berkas main.dart (atau tautan repositori)

Hasil Tugas (di folder Tugas 4)

```
→ tugas_4 git:(main) X flutter pub add http
   Woah! You appear to be trying to run flutter as root.
   We strongly recommend running the flutter tool without superuser privileges.
  /
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
+ http 1.6.0
+ http_parser 4.1.2
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  test_api 0.7.12 (0.7.15 available)
+ typed_data 1.4.0
+ web 1.1.1
Changed 4 dependencies!
4 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
→ tugas_4 git:(main) X
```

```dart
import 'dart:convert';

import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const MyApp());

class Post {
  final int id;
  final int userId;
  final String title;
  final String body;

  const Post({
    required this.id,
    required this.userId,
    required this.title,
    required this.body,
  });

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'] as int,
      userId: json['userId'] as int,
      title: json['title'] as String,
      body: json['body'] as String,
    );
  }
}

class Komentar {
  final int id;
  final int postId;
  final String name;
  final String email;
  final String body;

  const Komentar({
    required this.id,
    required this.postId,
    required this.name,
    required this.email,
    required this.body,
  });

  factory Komentar.fromJson(Map<String, dynamic> json) {
    return Komentar(
      id: json['id'] as int,
      postId: json['postId'] as int,
      name: json['name'] as String,
      email: json['email'] as String,
      body: json['body'] as String,
    );
  }
}
```

```dart
class PostService {
  static const String _baseUrl = 'https://jsonplaceholder.typicode.com';

  static Future<List<dynamic>> _getList(String path) async {
    final uri = Uri.parse('$_baseUrl$path');
    final response = await http.get(uri).timeout(const Duration(seconds: 10));
    if (response.statusCode != 200) {
      throw Exception('Gagal memuat data (kode ${response.statusCode})');
    }
    return jsonDecode(response.body) as List<dynamic>;
  }

  static Future<List<Post>> ambilPosts() async {
    final data = await _getList('/posts');
    return data.map((e) => Post.fromJson(e as Map<String, dynamic>)).toList();
  }

  static Future<List<Komentar>> ambilKomentar(int postId) async {
    final data = await _getList('/posts/$postId/comments');
    return data
        .map((e) => Komentar.fromJson(e as Map<String, dynamic>))
        .toList();
  }
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Daftar Postingan',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const PostListPage(),
    );
  }
}

class TampilanError extends StatelessWidget {
  final Object? error;
  final VoidCallback onRetry;

  const TampilanError({super.key, required this.error, required this.onRetry});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Icon(Icons.error_outline, size: 48, color: Colors.red),
            const SizedBox(height: 8),
            Text(
              'Terjadi kesalahan:\n$error',
              textAlign: TextAlign.center,
```



```
                ),
                const SizedBox(height: 12),
                ElevatedButton(
                  onPressed: onRetry,
                  child: const Text('Coba lagi'),
                ),
              ],
            ),
          ),
        );
      }
    }

    class PostListPage extends StatefulWidget {
      const PostListPage({super.key});

      @override
      State<PostListPage> createState() => _PostListPageState();
    }

    class _PostListPageState extends State<PostListPage> {
      late Future<List<Post>> _future;

      @override
      void initState() {
        super.initState();
        _future = PostService.ambilPosts();
      }

      void _muatUlang() {
        setState(() {
          _future = PostService.ambilPosts();
        });
      }

      @override
      Widget build(BuildContext context) {
        return Scaffold(
          appBar: AppBar(
            title: const Text('Daftar Postingan'),
            actions: [
              IconButton(icon: const Icon(Icons.refresh), onPressed: _muatUlang),
            ],
          ),
          body: FutureBuilder<List<Post>>(
            future: _future,
            builder: (context, snapshot) {
              if (snapshot.connectionState == ConnectionState.waiting) {
                return const Center(child: CircularProgressIndicator());
              }
              if (snapshot.hasError) {
                return TampilanError(error: snapshot.error, onRetry: _muatUlang);
              }

              final data = snapshot.data!;
              if (data.isEmpty) {
                return const Center(child: Text('Tidak ada data'));
              }
```



```
              return ListView.separated(
                itemCount: data.length,
                separatorBuilder: (_, __) => const Divider(height: 1),
                itemBuilder: (context, i) {
                  final p = data[i];
                  return ListTile(
                    title: Text(
                      p.title,
                      maxLines: 1,
                      overflow: TextOverflow.ellipsis,
                      style: const TextStyle(fontWeight: FontWeight.bold),
                    ),
                    subtitle: Text(
                      p.body.replaceAll('\n', ' '),
                      maxLines: 2,
                      overflow: TextOverflow.ellipsis,
                    ),
                    trailing: const Icon(Icons.chevron_right),
                    onTap: () {
                      Navigator.push(
                        context,
                        MaterialPageRoute(
                          builder: (_) => PostDetailPage(post: p),
                        ),
                      );
                    },
                  );
                },
              );
            },
          ),
        );
      }
    }

    class PostDetailPage extends StatefulWidget {
      final Post post;

      const PostDetailPage({super.key, required this.post});

      @override
      State<PostDetailPage> createState() => _PostDetailPageState();
    }

    class _PostDetailPageState extends State<PostDetailPage> {
      late Future<List<Komentar>> _futureKomentar;

      @override
      void initState() {
        super.initState();
        _futureKomentar = PostService.ambilKomentar(widget.post.id);
      }

      void _muatUlang() {
        setState(() {
          _futureKomentar = PostService.ambilKomentar(widget.post.id);
        });
      }
```



```
      @override
      Widget build(BuildContext context) {
        final post = widget.post;

        return Scaffold(
          appBar: AppBar(
            title: Text('Postingan #${post.id}'),
            actions: [
              IconButton(icon: const Icon(Icons.refresh), onPressed: _muatUlang),
            ],
          ),
          body: FutureBuilder<List<Komentar>>(
            future: _futureKomentar,
            builder: (context, snapshot) {
              // Bagian isi postingan selalu tampil di atas.
              final header = Padding(
                padding: const EdgeInsets.all(16),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      post.title,
                      style: Theme.of(context).textTheme.titleLarge,
                    ),
                    const SizedBox(height: 8),
                    Text(post.body),
                    const SizedBox(height: 16),
                    const Divider(),
                    Text(
                      snapshot.hasData
                          ? 'Komentar (${snapshot.data!.length})'
                          : 'Komentar',
                      style: Theme.of(context).textTheme.titleMedium,
                    ),
                  ],
                ),
              );

              Widget isiKomentar;
              if (snapshot.connectionState == ConnectionState.waiting) {
                isiKomentar = const Padding(
                  padding: EdgeInsets.all(32),
                  child: Center(child: CircularProgressIndicator()),
                );
              } else if (snapshot.hasError) {
                isiKomentar = TampilanError(
                  error: snapshot.error,
                  onRetry: _muatUlang,
                );
              } else if (snapshot.data!.isEmpty) {
                isiKomentar = const Padding(
                  padding: EdgeInsets.all(32),
                  child: Center(child: Text('Belum ada komentar')),
                );
              } else {
                isiKomentar = Column(
                  children: snapshot.data!
                      .map(
                        (k) => ListTile(
                          leading: CircleAvatar(child: Text(k.name[0].toUpperCase())),
                          title: Text(k.name),
                          subtitle: Text('${k.email}\n${k.body}'),
                          isThreeLine: true,
                        ),
                      )
                      .toList(),
                );
              }

              return ListView(children: [header, isiKomentar]);
            },
          ),
        );
      }
    }
```



